

```
[pmu@t430 mydotfiles]$ git ls-files --stage
100644 b70f72952f495b2aae83f2ffa50b5ee8d001edb 0    README.
txt
[pmu@t430 mydotfiles]$
```

No, it does not. So, let's go ahead and add the *Oncemore.txt* file. We will use a '.' this time so that it adds all the files (*Readme.txt* and *Oncemore.txt*).

```
[pmu@t430 mydotfiles]$ git add .

[pmu@t430 mydotfiles]$ git status
# On branch master
# Initial commit
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#   new file:   Oncemore.txt
#   new file:   Readme.txt
[pmu@t430 mydotfiles]$
```

Note how the *Oncemore.txt* file now shows up as a new file instead of as 'untracked'. So, how does Git see these two files? Let's find out again with the *Git ls-files --stage* command.

```
[pmu@t430 mydotfiles]$ git ls-files --stage
100644 b70f72952f495b2aae83f2ffa50b5ee8d001edb 0    Oncemore.
txt
100644 b70f72952f495b2aae83f2ffa50b5ee8d001edb 0    README.
txt
[pmu@t430 mydotfiles]$
```

Well, it's basically two files with different names, but with the same checksum.

The following output confirms it:

```
[pmu@t430 mydotfiles]$ git ls-files --stage | awk {'print $2'}
| sort | uniq
b70f72952f495b2aae83f2ffa50b5ee8d001edb
[pmu@t430 mydotfiles]$
```

Indeed, the checksum value is the same; it's just that there are two file names associated with it. The *git show* command lets us peek into the file using the checksum value. If you and I were to see the file content, we would use *cat/more/less* or something like that. But Git uses the *git show* command and we supply the checksum to it.

```
[pmu@t430 mydotfiles]$ git show b70f
This is a Readme file, my first file that I'll try to upload on
GitHub.
[pmu@t430 mydotfiles]$
```

If you were expecting to see the content mentioned twice, you're still thinking the 'filename

way' which is not the case here. For Git, it's just *b70f72952f495b2aae83f2ffa50b5ee8d001edb*. This is what Git calls a blob.

Let's leave *Oncemore.txt* alone for a while now and just focus on *Readme.txt*.

We'll now run the *git commit* command, which will add the file to the Git local repository.

```
@t430 mydotfiles]$ git commit -m "Adding my first file - README.
txt" README.txt
[master (root-commit) 26ab994] Adding my first file - README.
txt
1 file changed, 1 insertion(+)
create mode 100644 README.txt
[pmu@t430 mydotfiles]$
```

The output shows that the *Readme.txt* file is now committed to the repository. The part added in quotes after *-m* is the message or comment. The output above tells us that there is one insertion and the last three digits that appear after the 'create mode' word - 644 indicate the 'umask' of the file. So what's the number 26ab994? Let's find out using the *git show* command once again:

```
[pmu@t430 mydotfiles]$ git show 26ab994
commit 26ab994663499b21d8e2de7fc0f53925954fae7c
Author: pmu <pmu.rwx@gmail.com>
Date: Mon Sep 22 21:08:18 2014 +0530
    Adding my first file - README.txt
diff --git a/Readme.txt b/Readme.txt
new file mode 100644
index 0000000..b70f729
--- /dev/null
+++ b/Readme.txt
@@ -0,0 +1 @@
+This is a Readme file, my first file that I'll try to upload on
+GitHub.
[pmu@t430 mydotfiles]$
```

That is the commit object hash checksum. This is the Git object which has the author's name, the date and time, the comment, the hashed checksum name of the file that was committed, and some more information. Note that in the line 'index 0000000..b70f729', the last seven digits are nothing but the first seven digits of the checksum that Git created for our *Readme.txt* file.

Let's run the *git status* command again.

```
[pmu@t430 mydotfiles]$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#   new file:   Oncemore.txt
[pmu@t430 mydotfiles]$
```